
Machine Learning Analysis of Housing Price Predictors:

A Comparative Study Using the Ames Housing Dataset

Alena Thomas
NetID: amt325
CS439 Section 08

James Booth
NetID: jpb302
CS439 Section 08

Abstract

Predicting housing prices accurately is a problem that shows up everywhere from mortgage lending to home buying decisions, so we thought it would be a good fit for a data science project. We looked at which housing features most affect sale prices and compared four regression models on the Ames Housing dataset, which has 1,460 homes and 79 features covering everything from square footage to garage quality. We did standard preprocessing, log-transforming the sale price, converting categorical variables to numbers, and filling in any remaining missing values, then trained Linear Regression, Ridge, Lasso, and Random Forest models and evaluated them on a held-out 20% test set using RMSE and MAE. Random Forest ended up being the best model with an RMSE of 0.1472 on the log-transformed price. Looking at which features mattered most, overall quality rating came out overwhelmingly on top with an importance score of 0.5475, followed by above-ground living area at 0.1135, both way ahead of anything else. Ridge and Lasso both improved on basic linear regression, Lasso especially, since it automatically removed a lot of the weaker features. Overall, this project showed us that more complex models can outperform simple ones on real-world data, and that looking at feature importance can be just as valuable as looking at prediction accuracy.

Code: <https://github.com/JayPeeBee825/cs439-final-project.git>
Video: <https://1drv.ms/v/c/a5c16cfa7acd0581/IQD12dtXM8fKR6K1tqW-nViIAZq6Ocki4xmrJnMcn5n0NG4?e=0BoFQV>

1 Introduction

Housing prices are something most people think about at some point in their lives, whether it's figuring out what you can afford to buy, understanding why rent is so high, or just being curious why one house costs twice as much as the one next door. Despite being such a relatable problem, predicting home values accurately is actually pretty hard. A house's price depends on dozens of factors at once: its size, age, condition, location, how many bathrooms it has, whether the basement is finished, and so on. Some of these matter a lot and some barely matter at all.

For our class project, we wanted to use machine learning to tackle this problem. We chose the Ames Housing dataset (2), originally put together by De Cock (1), because it's one of the most well-known datasets for regression tasks and has a large variety of features to work with. The dataset covers home sales in Ames, Iowa and has 79 features describing almost every aspect of each property.

We had two main goals going into this project:

- 34 1. **Which features matter most?** We wanted to figure out which housing characteristics are the
35 best predictors of sale price.
- 36 2. **Which model works best?** We wanted to compare a few different regression approaches, from
37 simple linear models to more complex ones, and understand why some perform better than
38 others.

39 We thought both questions were worth exploring because they answer different things. The model
40 comparison tells you which approach to use in practice. The feature importance analysis tells
41 you something interesting about the housing market itself, which factors actually drive price. A
42 homeowner deciding whether to renovate their kitchen versus finish their basement, for instance,
43 might make a very different choice depending on what the data says.

44 2 Related Work

45 **Traditional approaches to housing prices.** Economists have studied housing prices for a long time
46 using something called a hedonic pricing model (3), which basically means fitting a linear regression
47 where the features are things like square footage, number of bedrooms, and neighborhood. This
48 approach is still used today because it's simple and easy to interpret. The downside is that it assumes
49 a straightforward linear relationship between each feature and price, which isn't always true. For
50 example, the value of an extra bathroom probably depends on how big the house already is, and a
51 simple linear model can't capture that kind of interaction.

52 **Regularized regression.** When you have a lot of features, regular linear regression tends to overfit,
53 it starts learning patterns specific to the training data that don't hold up on new examples. Two
54 common fixes for this are Ridge regression (4) and Lasso regression (5). Ridge works by shrinking
55 all the feature coefficients toward zero, reducing the model's tendency to overfit. Lasso goes further
56 and can set some coefficients all the way to zero, which effectively removes those features from the
57 model entirely. This is called automatic feature selection, and it's especially useful when you suspect
58 many of your features aren't actually helpful.

59 **Tree-based models.** More recently, models like Random Forest (7) and gradient boosted trees
60 (such as XGBoost (8) and LightGBM (9)) have become very popular for structured prediction tasks
61 like this one. These models build many decision trees and combine their predictions, which allows
62 them to capture non-linear patterns and interactions between features automatically, without you
63 having to manually create those combinations. On the Ames Housing Kaggle competition, most of
64 the top-performing solutions use gradient boosting. We used Random Forest since it's a natural and
65 interpretable first step toward those more complex methods.

66 **Understanding model predictions.** Beyond just accuracy, there's a lot of interest in understanding
67 *why* a model makes the predictions it does. Random Forest has a built-in way to measure feature
68 importance based on how much each feature helps reduce prediction error across all the trees. More
69 advanced techniques like SHAP values (10) give even more detailed explanations, breaking down
70 each prediction individually. For this project we used the built-in importance scores since they were
71 straightforward and easy to explain.

72 3 Method

73 3.1 Dataset

74 We used the Ames Housing dataset (2), which contains 1,460 homes and 79 features describing
75 residential property sales in Ames, Iowa from 2006 to 2010. The features cover a wide range of
76 information about each home:

- 77 • **Size:** Above-ground living area (`GrLivArea`), basement area (`TotalBsmtSF`), first and second
78 floor square footage, garage area, and lot area.
- 79 • **Quality and condition:** Overall quality (`OverallQual`) and overall condition (`OverallCond`),
80 both rated on a 1–10 scale, plus quality ratings for specific parts of the home like the kitchen
81 and exterior.

- **Room counts:** Number of bedrooms, bathrooms (full and half, above and below grade), and total rooms above grade.
- **Age and renovations:** Year the home was built (YearBuilt) and year of the most recent remodel (YearRemodAdd).
- **Garage:** Garage type, how many cars it fits (GarageCars), and total garage area (GarageArea).
- **Neighborhood:** Categorical variables for neighborhood and zoning.
- **Other:** Pool quality, fence type, fireplace count, sale type, and more.

The target variable we're trying to predict is `SalePrice`, which ranges from about \$35,000 to \$755,000 with a mean around \$181,000. Figure 1 shows the raw distribution, most homes are in the \$100k–\$250k range, but there's a long tail of expensive properties. That skew was one of the first things we had to deal with in preprocessing.

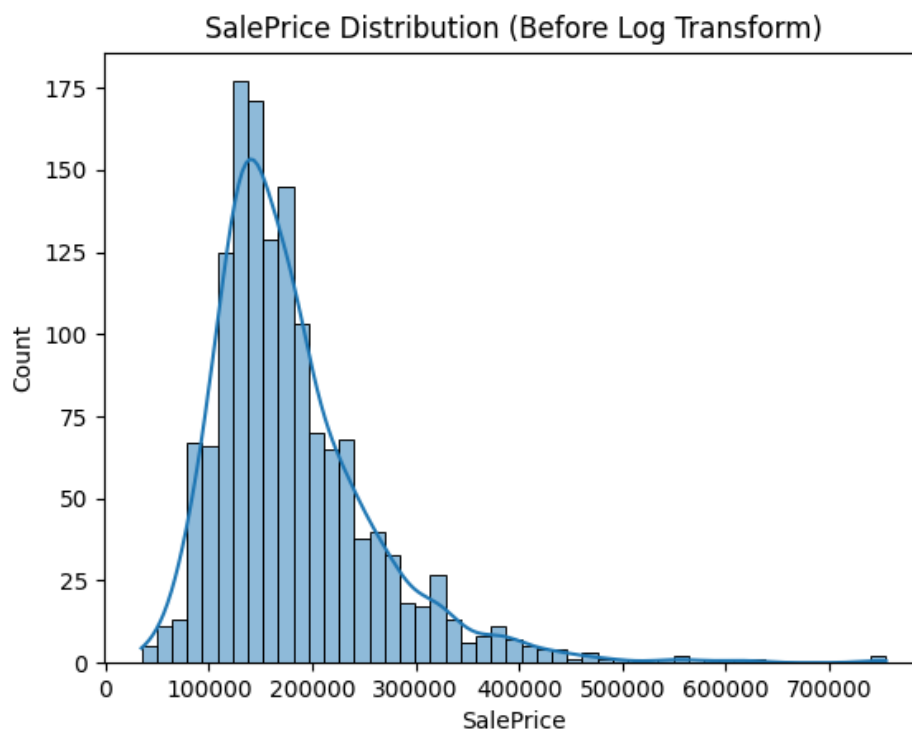


Figure 1: Distribution of `SalePrice` before any transformation. The distribution is right-skewed with a long tail of high-priced homes, which is why we applied a log transformation before training our models.

3.2 Preprocessing

Before we could train any models, we needed to clean and prepare the data. This step ended up being more involved than we expected, but the choices we made here had a real impact on model performance.

Log-transforming the sale price. Because the raw prices are skewed (as shown in Figure 1), we applied a log transformation: $y = \log(1 + \text{SalePrice})$. This makes the distribution much more symmetric and reduces the influence of very expensive outlier homes during training. All of our accuracy metrics (RMSE and MAE) are calculated on this log scale, so an error of 0.15 corresponds roughly to a 15% error on the actual dollar price.

Converting categorical variables. We converted all categorical features to numbers using dummy encoding, which creates a separate yes/no column for each possible category value. This expanded our feature set from 79 original features to 245 columns. Some features like `Neighborhood` (which has 25 possible values) contribute a lot of columns on their own, and most of those columns will be zero for any given home, which is part of why feature selection becomes so important.

Splitting into training and test sets. We used 80% of the data for training (1,168 homes) and held out 20% for testing (292 homes), using a fixed random seed of 42 so our results are reproducible. For Ridge and Lasso, we used fixed regularization strengths of $\lambda = 1.0$ for Ridge and $\lambda = 0.001$ for Lasso, which are standard starting values commonly used in practice. The test set was never used until we had fully trained each model.

Handling missing values. Some features had missing values. Since we converted all categorical variables to numeric dummy columns before filling in missing values, we handled everything with a single approach: replacing any remaining missing values with the median of each column, calculated from the training data only, and then applied to the test set. This ensures no information from the test set influenced how we filled in the training data.

Scaling features. For the three linear models, we standardized all numerical features so they have a mean of zero and a standard deviation of one. This is important for Ridge and Lasso because the penalty they apply to coefficients needs to treat all features equally, without scaling, features measured in large units (like square footage) would be penalized unfairly compared to features measured in small units. Random Forest doesn't need this since it only looks at whether one value is bigger or smaller than another, not the actual numbers.

3.3 Models

We trained four models, going from the simplest to the most complex.

Linear Regression. This is the most basic approach: find the line (or in this case, a plane in high-dimensional space) that best fits the training data by minimizing the total squared error:

$$\hat{\beta} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2.$$

We included this mainly as a baseline. With 245 features and only about 1,200 training examples, it was likely to overfit.

Ridge Regression. Ridge adds a penalty term to the linear regression objective that discourages large coefficients:

$$\hat{\beta}^{\text{ridge}} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_2^2.$$

The strength of the penalty is controlled by λ , which we set to $\lambda = 1.0$. Ridge keeps all features in the model but shrinks their coefficients, making the model less sensitive to noise.

Lasso Regression. Lasso uses a slightly different penalty that can shrink some coefficients all the way to zero, effectively removing those features from the model:

$$\hat{\beta}^{\text{lasso}} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_1.$$

We chose $\lambda = 0.001$. We were especially interested in Lasso because, after dummy encoding, we suspected a lot of our 245 features were not actually useful, and Lasso can automatically identify and drop those.

Random Forest. Random Forest works by training a large number of decision trees, each on a slightly different random sample of the training data and using a random subset of features at each split (7). The final prediction is the average of all the individual tree predictions. This averaging is what makes Random Forest more stable and accurate than any single decision tree. We used 100 trees with the standard default settings. The key advantage over linear models is that Random Forest can capture non-linear patterns and interactions between features without us having to manually engineer them.

145 4 Evaluation

146 4.1 Metrics

147 We measured performance using two metrics on the held-out test set:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}, \quad \text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|.$$

148 RMSE (Root Mean Squared Error) penalizes large mistakes more heavily because of the squaring, a
149 prediction that's way off hurts the score more than several small errors. MAE (Mean Absolute Error)
150 just averages all the errors equally, making it easier to interpret as a straightforward “how wrong were
151 we on average.” We report both because they can tell different stories: a model with better RMSE but
152 worse MAE is mainly better at avoiding the really bad predictions, not at being more accurate overall.

153 4.2 Results

154 Table 1 shows how all four models performed on the test set.

Table 1: Test set performance of all four models on log-transformed `SalePrice`. Lower values are better for both metrics.

Model	RMSE	MAE
Linear Regression	0.2168	0.0972
Ridge Regression	0.1998	0.0967
Lasso Regression	0.1636	0.0970
Random Forest	0.1472	0.1001

155 **Linear Regression.** Plain OLS had the worst RMSE at 0.2168. This was expected, with 245
156 features and only about 1,200 training examples, it almost certainly memorized noise in the training
157 data that didn't generalize. One interesting thing: the MAE values are actually pretty similar across
158 all four models, which tells us that plain linear regression isn't dramatically worse at the *typical*
159 prediction, it just makes more extreme mistakes in certain cases, and those get amplified in RMSE.

160 **Ridge Regression.** Ridge improved RMSE to 0.1998. Shrinking the coefficients helped reduce
161 overfitting, but since Ridge keeps all features in the model, it's still working with a lot of potentially
162 useless columns. The improvement over OLS was real, but not as large as we got from Lasso.

163 **Lasso Regression.** Lasso was the best of the linear models with RMSE at 0.1636 and the second
164 lowest MAE at 0.0970. In our runs, it kept only around 60–80 features out of 245, throwing out the
165 rest. This makes a lot of sense: after dummy encoding, you end up with columns like “sale condition
166 = abnormal” that only apply to a handful of homes. Lasso correctly identifies those as unhelpful and
167 removes them, leaving a much cleaner model. The improvement from Ridge to Lasso was bigger
168 than we expected and suggested that the main problem wasn't just noisy coefficients, it was features
169 that flat-out shouldn't be in the model.

170 **Random Forest.** Random Forest had the best RMSE at 0.1472, about 10% better than Lasso and
171 32% better than plain OLS. Something interesting: its MAE (0.1001) is actually slightly worse than
172 Lasso's (0.0970). This means Random Forest's main advantage is in avoiding really bad predictions,
173 not necessarily in being more accurate on a typical home. That makes sense for a group, averaging
174 over 100 trees smooths out the extreme mistakes, even when the middle-of-the-road prediction is
175 similar to a good linear model.

176 4.3 Feature Importance

177 One of the things we were most curious about was which features the Random Forest relied on most.
178 We measured this using a metric called mean decrease in impurity, basically, how much each feature

179 helps reduce prediction error when it's used as a split point in a tree, averaged across all 100 trees.
180 Table 2 shows the top 10, and Figure 2 plots them visually.

Table 2: Top 10 most important features according to the Random Forest. Scores represent how much each feature reduces prediction error (higher = more important), and sum to 1 across all features.

Feature	Importance Score
OverallQual (Overall Quality Rating)	0.5475
GrLivArea (Above-Ground Living Area)	0.1135
TotalBsmtSF (Total Basement Area)	0.0442
GarageCars (Garage Capacity)	0.0416
GarageArea (Garage Square Footage)	0.0242
BsmtFinSF1 (Finished Basement Area)	0.0217
1stFlrSF (First Floor Area)	0.0195
CentralAir_Y (Has Central A/C)	0.0170
LotArea (Lot Size)	0.0152
YearBuilt (Year Built)	0.0147

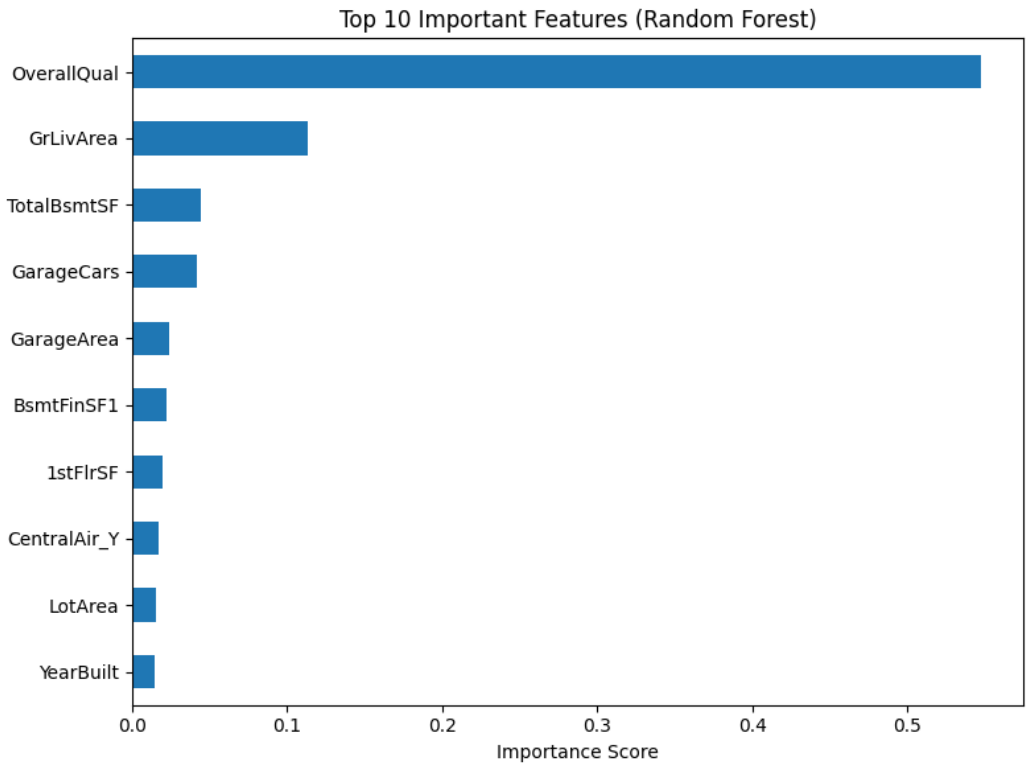


Figure 2: Top 10 feature importances from our Random Forest model. OverallQual dominates with a score of 0.5475, nearly five times higher than the second most important feature.

181 The most striking thing about Figure 2 is how much OverallQual stands out. With a score of 0.5475,
182 a single 10-point quality rating accounts for more than half of the model's total feature importance
183 across all 245 columns. That's a much stronger result than we expected. It suggests the model is
184 essentially first sorting homes into quality tiers, then using other features to fine-tune predictions
185 within each tier.

186 GrLivArea is clearly second at 0.1135, which makes intuitive sense, buyers pay for square footage.
187 After those two, there's a big drop-off, with basement area, garage capacity, and other size-related
188 features clustered between 0.02 and 0.045.

189 One surprise: CentralAir_Y (a simple yes/no flag for whether the house has central air conditioning)
190 ranked 8th. We didn't expect a binary feature like that to make the top 10, but it suggests that in
191 Ames during 2006–2010, a lot of homes didn't have central air, making it a meaningful differentiator.

192 Another interesting finding: neighborhood features didn't make the top 10 at all. In most housing
193 markets you hear "location, location, location," but in this dataset structural quality and size seem to
194 matter more than which neighborhood a home is in. This might be specific to Ames being a smaller,
195 relatively uniform college town.

196 5 Discussion

197 **More complex models do help, but so does a better linear model.** Random Forest was our best
198 model, which confirms the general idea that non-linear relationships in housing data are real and
199 worth capturing. What surprised us more was that the improvement from fixing the linear model
200 (OLS to Lasso: RMSE 0.2168 to 0.1636) was actually larger than the improvement from switching to
201 Random Forest (Lasso to Random Forest: 0.1636 to 0.1472). So before you even think about using a
202 fancier model, it's worth making sure your linear model is properly regularized. That was a practical
203 lesson we took away from this.

204 **Removing features matters more than just shrinking coefficients.** Among the linear models,
205 Lasso beat Ridge by more than we expected, and we think it's because Ridge's approach, keeping all
206 features but making their coefficients smaller, still leaves a lot of noise. After dummy encoding, many
207 columns represent rare combinations that appear in only a handful of training examples. Lasso's
208 ability to remove those entirely turns out to be more useful than just down-weighting them.

209 **One quality rating drives almost everything.** The 0.5475 importance score for OverallQual
210 was the most striking finding in this project. One single rating accounts for more than half of the
211 model's total feature importance. Part of what makes this interesting is that this is a subjective rating
212 assigned by whoever compiled the dataset, it compresses a lot of hard-to-measure things (material
213 quality, craftsmanship, finishes) into a single number. For prediction purposes this is great, it's an
214 incredibly useful signal. But it also means you can't necessarily use this model to make causal claims
215 like "improving quality by one point causes prices to rise by X dollars," because that rating might be
216 defined partly in terms of price to begin with.

217 **Central A/C as a surprise finding.** We didn't go into this project expecting central air conditioning
218 to be in the top 10 most important features. Looking back at the data, a noticeable fraction of homes
219 in the dataset lack central air, which means having it is a real differentiator in this market. It's a good
220 reminder that feature importance analysis can surface things you genuinely wouldn't have predicted
221 from intuition alone, which is part of what makes it a useful tool.

222 **Where the models struggle.** Looking at the predictions informally, all four models tended to
223 underestimate prices for the most expensive homes (roughly above \$400k) and overestimate prices for
224 some of the cheapest, most distressed properties. Very expensive homes often have unusual features,
225 custom materials, large lots, unique layouts, that appear rarely in the training data. This kind of
226 underprediction at the high end is a common challenge in regression problems with skewed targets.

227 6 Limitations

228 **This dataset is from one city and one time period.** All of our results are based on homes sold in
229 Ames, Iowa between 2006 and 2010. Ames is a relatively small, homogeneous college town, and our
230 findings might not transfer to a big city like Chicago or New York where location effects are much
231 stronger and the housing stock is more varied. The time window also overlaps with the 2008 financial
232 crisis, which we didn't account for at all.

No real location data. Even though we have neighborhood labels, there's no actual geographic data in this dataset, no coordinates, no distance to schools or transit, no information about local amenities. Location is widely known to be one of the most important factors in housing prices, and the fact that neighborhood features didn't rank highly in our model might partly reflect that our location data just isn't detailed enough, not that location genuinely doesn't matter.

We didn't tune the Random Forest. We used the default settings for Random Forest throughout (100 trees, square root of features per split, no depth limit). Spending time tuning these settings would likely improve performance. We also never tried gradient boosting methods like XGBoost, which tend to outperform Random Forests on structured data like this and would probably push the accuracy further.

Dummy encoding creates a lot of noise. Converting categorical features to binary columns is a standard approach, but it creates many sparse columns, especially for features with many categories. A technique called target encoding, where you replace each category with the average sale price for homes in that category, can work better. This is probably one reason Ridge did worse than Lasso: it kept all those sparse columns instead of discarding them.

Feature importance scores have a known bias. The impurity-based importance scores we used are known to favor features with more unique values (11). Since `OverallQual` has 10 possible values, its extremely high score might be slightly inflated compared to binary features like `CentralAir_Y`. A method called permutation importance, or SHAP values, would give more reliable rankings and is something we'd want to use if we continued this work.

7 Conclusion

In this project we trained and compared four regression models, Linear Regression, Ridge, Lasso, and Random Forest, on the Ames Housing dataset to predict sale prices and understand which features matter most.

Random Forest was our best model with an RMSE of 0.1472, about 10% better than Lasso and 32% better than plain linear regression. This confirmed that housing prices involve non-linear patterns that simpler models miss. Among the linear models, Lasso was significantly better than both plain regression and Ridge, which showed us that automatically removing unhelpful features is more powerful than just shrinking their coefficients.

The feature importance results, shown in Figure 2, were probably the most interesting part of the project. `OverallQual` alone accounted for over half of the Random Forest's total feature importance, which we didn't expect. Living area was a clear second, and the rest of the top 10 was mostly size-related features. Neighborhood didn't make the top 10 at all, and central air conditioning did, both were somewhat surprising. The consistency between what Random Forest and Lasso flagged as important gave us more confidence that these findings reflect something real about the data.

If we were to extend this project, the most useful next steps would be trying gradient boosted trees, adding geographic data like coordinates or proximity to amenities, and using SHAP values for a more rigorous feature importance analysis. Overall though, this was a good exercise in going through a complete data science pipeline, from messy raw data to trained models to results we can actually interpret and learn from.

References

- [1] D. De Cock. Ames, Iowa: Alternative to the Boston housing data as an end of semester regression project. *Journal of Statistics Education*, 19(3), 2011.
- [2] Kaggle. House Prices: Advanced Regression Techniques. <https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques>, 2016.
- [3] S. Rosen. Hedonic prices and implicit markets: Product differentiation in pure competition. *Journal of Political Economy*, 82(1):34–55, 1974.

- 280 [4] A. E. Hoerl and R. W. Kennard. Ridge regression: Biased estimation for nonorthogonal
281 problems. *Technometrics*, 12(1):55–67, 1970.
- 282 [5] R. Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical
283 Society: Series B*, 58(1):267–288, 1996.
- 284 [6] H. Zou and T. Hastie. Regularization and variable selection via the elastic net. *Journal of the
285 Royal Statistical Society: Series B*, 67(2):301–320, 2005.
- 286 [7] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- 287 [8] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd
288 ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages
289 785–794, 2016.
- 290 [9] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A
291 highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing
292 Systems*, volume 30, 2017.
- 293 [10] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *Advances
294 in Neural Information Processing Systems*, volume 30, 2017.
- 295 [11] C. Strobl, A.-L. Boulesteix, A. Zeileis, and T. Hothorn. Bias in random forest variable impor-
296 tance measures: Illustrations, sources and a solution. *BMC Bioinformatics*, 8(1):25, 2007.
- 297 [12] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel,
298 P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher,
299 M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine
300 Learning Research*, 12:2825–2830, 2011.
- 301 [13] G. James, D. Witten, T. Hastie, and R. Tibshirani. *An Introduction to Statistical Learning*.
302 Springer, New York, 2013.
- 303 [14] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*, 2nd ed. Springer,
304 New York, 2009.
- 305 [15] J. H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of
306 Statistics*, 29(5):1189–1232, 2001.
- 307 [16] A. Liaw and M. Wiener. Classification and regression by randomForest. *R News*, 2(3):18–22,
308 2002.
- 309 [17] D. De Cock. Ames housing dataset documentation. [http://jse.amstat.org/v19n3/
310 decock/DataDocumentation.txt](http://jse.amstat.org/v19n3/decock/DataDocumentation.txt), 2011.